

Lab 06 - Character Reference Sheets and Consistency Features

Course: Create Short Video Film using AI (TGS-2020505925) | Version v12.0 | 27 September 2026 Topic 03: AI Visual Features, Character Consistency and Scene Design Outcome: ELO3 | In-class time: 30 minutes of scheduled practical time

Competency	Statement
Ability A4	Design and implement feature extraction and representation methods
Knowledge K5	Feature extraction and representation techniques
Knowledge K6	Local feature descriptions, edge, colour, texture and motion

Scenario

MEI appears in six generated shots. Three of them have drifted from her reference sheet in ways a tired reviewer can miss. You design a consistency check from local features - colour, edge, texture and ORB keypoints - implement it on the character and prop crops, set the thresholds, and prove which shots must be regenerated.

What you will produce

A feature-extraction design (which feature detects which failure), an identity report for six shots with pass/drift verdicts and reasons, and an identity strip comparing each shot crop with the reference sheet.

Tools: Python 3 | opencv-python | NumPy | optional: an image model with character references

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: opencv-python>=4.10, numpy.

API currency. Everything taught here runs on the base opencv-python wheel plus NumPy - no contrib modules and no downloaded model weights. SSIM is implemented in NumPy because cv2.quality is contrib-only. Generative steps are optional: every lab supplies SIMULATED reference media so it can be completed offline, and asks you to record the model ID and date if you generate your own.

Workflow

1. Study the reference sheet as a feature specification
2. Choose a feature per failure mode
3. Extract colour, edge, texture and ORB features

- Set thresholds and run the check
- Decide which shots to regenerate

Files in this folder

Path	What it is
reference/mei-reference-sheet.png	MEI reference sheet: front, side, back, close-up and the RED BOX (SIMULATED)
reference/shots/shot_01.png ... shot_06.png	Six shots of MEI, three with injected drift (SIMULATED)
data/crops.json	Character, prop and clasp boxes per shot (generated with the frames)
data/identity-thresholds.json	Colour ranges and starting thresholds
character_features.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom illustration of *The Last Ferry*, rendered deterministically with OpenCV and labelled on the frame, or an AI-generated reference copied unchanged with a PROVENANCE.md beside it. Nothing in this folder may be relabelled: a simulated stand-in is never presented as model output or as a photograph.

Steps

- Open the reference sheet. List every feature a viewer uses to recognise MEI: yellow hooded raincoat, red scarf, black bag with a chequered patch, the RED BOX with a brass clasp. This list is your feature specification.
- Complete a design table in out/feature-design.md with three columns: failure mode (coat colour drift, scarf change, lost prop detail, different person), the feature that detects it, and why that feature is repeatable across shots.
- Read data/crops.json. Each shot has a character box, a prop box and a clasp box. In production these boxes come from the Lab 08 detector or a manual mark-up.
- Extract the coat colour of the sheet crop in HSV. OpenCV 8-bit hue runs 0-179, so yellow sits near 22-23.

```
python3 -c "import cv2,json,numpy as np;s=cv2.imread('reference/mei-reference-sheet.png');x,y,w,h=json.load(open('data/crops.json'))['crops']['sheet']['character'];hsv=cv2.cvtColor(s[y:y+h,x:x+w],cv2.COLOR_BGR2HSV);m=cv2.inRange(hsv,(5,150,170),(40,255,255))>0;print(np.median(hsv[...0][m]))"
```

- Run the full check. It extracts colour, k-means dominant colours, Canny edge density, Laplacian texture energy and ORB keypoint matches for every shot.

```
python3 character_features.py --sheet reference/mei-reference-sheet.png --shots reference/shots --crops data/crops.json --thresholds data/identity-thresholds.json --out out
```

- Read the verdicts. Expect DRIFT on shot_04 (scarf), shot_05 (clasp missing) and shot_06 (coat hue 13 instead of 23). Match each to the feature that caught it.
- Look at orb_good_matches: shot_06 still matches the sheet with about 12 good matches. Explain why a keypoint match on the bag patch cannot see a colour change.
- Look at the brass measurement. The raincoat and the brass clasp share a hue, which is why the check measures brass only inside the small clasp window. Record this as a design decision.

9. Tighten coat_hue_tolerance from 4 to 1 and rerun. Does any good shot now fail? Restore the value and write down the tolerance you would ship and why.
10. Open out/identity-strip.png (sheet first, red = drift) and check the verdicts by eye.
11. Write the regeneration list in out/regenerate.md: shot, failed feature, and the prompt or reference fix (for example: re-attach the sheet; add 'red scarf exactly as reference' to constraints).
12. Optional extension: save your own generated shots of a character with a reference sheet, add their boxes to crops.json and run the same check.

Verify

shot_01, shot_02 and shot_03 pass; shot_04, shot_05 and shot_06 each fail on exactly the feature designed to catch them.

Expected outputs

- out/feature-design.md
- out/identity-report.csv
- out/identity-strip.png
- out/regenerate.md

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- ☐ Feature-design table mapping each failure mode to a feature
- ☐ Identity report with verdicts and reasons for all six shots
- ☐ Threshold experiment with the tolerance you would ship
- ☐ Regeneration list with a prompt or reference fix per failed shot

Troubleshooting

Symptom	What to do
coat hue is nan	No pixel fell in the coat HSV range: check the crop box and the ranges in identity-thresholds.json.
ORB finds 0 matches	Crops must contain texture; flat colour has no corners. Include the bag patch or the face.
A good shot fails on brass	Check the clasp box is on the clasp; lower min_clasp_brass_cover and justify it.
k-means results change slightly between runs	k-means uses random seeds; the dominant colour, not its exact value, is the feature.

References used in this lab

- [S6] Youri van Hofwegen, 'EXACTLY How to Make an AI Short Film (Full Workflow)', YouTube, 1 Jun 2026, youtube.com/watch?v=0v534yAyhwg - *Full video, 16:49*.

Reference-first principle (stop giving the model freedom); three reference types - characters, locations, props; casting attributes (genre, budget, era, archetype, identity, physique, details, outfit); a second, damaged-state location made by editing the first ('change nothing else'); director controls (genre, camera movement, speed ramp,

duration, aspect ratio, audio); per-character emotion and intensity; the multi-shot prompt framework with dialogue in quotation marks; video reference versus start/end frame (mood carries over); three generations for dense action shots; assembly in a timeline editor.

- [S7] Mira AI, '6 AI Video Tools I'd Use To Make An AI Short Film', YouTube, 14 Jun 2026, youtube.com/watch?v=Wwh8vCqboxQ - *Full video, 10:04*.

A multi-tool workflow: an LLM to plan the story shot by shot, a cinematic image model for character sheets (front, side, back, close-ups) and multi-angle environment grids, a design canvas for the storyboard with keyframe placeholders, a video model for motion, an AI music generator (instrumental, mood and tempo), and an editor; the six-point prompt framework (subject, action, environment, camera, style, constraints); timeline prompting; video extension to pass the previous clip; generations capped at about 15 seconds.

- [S9] Roboverse, 'The Complete Guide to Making Cinematic AI Videos (2026)', YouTube, 19 Jul 2026, youtube.com/watch?v=HMVSCEh72n4 - *Chapters, 10:30*.

Planning the story with an LLM; developing the narrative; creating consistent assets; staging the visual scenes; cinematic visual grading for a film look before animation; refining motion and prompts; generating clips; post-production and finishing.

- [S17] OpenCV 4.13 Python package and FFmpeg, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92); ffmpeg/ffprobe optional*.

Every lab command runs on the base opencv-python wheel plus NumPy: VideoCapture / VideoWriter, cvtColor, calcHist/compareHist, Canny, ORB_create + BFMatcher, HuMoments, inpaint, CLAHE, filter2D, fastNIMeansDenoisingColored, Haar cascades from cv2.data.haarcascades, HOGDescriptor_getDefaultPeopleDetector, CamShift and calcOpticalFlowFarneback. SSIM is implemented in NumPy because cv2.quality is contrib-only. FFmpeg is used only when present, to mux audio into the final export.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v12.0.